CS3604 12306 项目开发报告

成员：陈卓妍 523030910
李青雅 523030910004
侯湘贻 523030910

2026 年 1 月 19 日

目录

1 项目简介 **3**

2 需求管理 **3**

2.1 从代码和 12306 页面反推需求 3

2.2 用 YAML 文件代替传统文档 3

2.3 进度追踪 4

3 需求规模 **5**

3.1 顶层模块数量 5

3.2 子需求和场景数量 5

4 智能体辅助设计与测试 **6**

4.1 MCP 工具链的使用 6

4.2 AI 怎么帮我们干活 6

4.3 如何保证可复现 7

5 测试用例与通过率 **7**

5.1 测试情况 7

5.2 测试结果 7

6 总结 **8**

1 项目简介

本项目基于课程提供的 12306 基础代码，在有限时间内完成了一个可运行、可复现的购票系统原型，涵盖了“查票—下单—支付—餐饮”的主要使用场景。

我们的主要工作不仅仅是“写代码”，更重要的是尝试了一套新的开发模式：

1. **用代码管理需求**：把需求写成 YAML 文件，而不是 Word 文档。
2. **让智能体辅助开发**：利用 MCP 工具链，让 AI 能够理解我们的需求队列，并辅助我们做模块设计和测试撰写。
3. **保证可复现性**：通过固定的数据库快照（sqlite），确保任何人（包括老师和助教）拿到代码后，都能看到一样的车次和数据，复现一样的功能。

本报告围绕老师提出的四个问题展开：

- 我们是**如何进行需求管理**的；
- 当前系统的**需求规模**大致到什么程度；
- 我们是如何**用智能体来完成模块设计和测试撰写**的；
- 目前**测试用例和通过情况**如何。

2 需求管理

2.1 从代码和 12306 页面反推需求

一开始我们没有从零写一份很厚的 SRS，而是先做了两件事：

- 通读课程给的前后端代码，看现有实现到底支持哪些功能。
- 对照真正的 12306 网站，梳理出“首页—登录—查票—下单—支付—个人中心—忘记密码—餐饮”的主流程。

在这个基础上，我们把“已经实现的功能”和“仍然想做的功能”整理成一棵需求树，再用代码的形式落在 YAML 里。

2.2 用 YAML 文件代替传统文档

传统的软件工程作业中，需求文档（SRS）往往写完就丢在一边，和代码严重脱节。为了解决这个问题，我们建立了一棵“活着的需求树”，存放在项目根目录的 `requirements.yaml` 文件中。

这个文件主要包含：

- **需求节点**：每个功能点都有唯一的 ID（如 `REQ-1-1`、`REQ-2-2-1`）。
- **场景描述**：每个功能点下都有具体的 Scenario，描述用户怎么操作（Action）以及系统应该有什么反应（Expectation）。
- **UI 规格**：通过 `ui_spec` 字段，给出页面路径、分区结构以及细致到像素级的 UI 描述。

- **资源引用**：在描述里直接关联参考图片（如 `./reference/auth/login.png`）和静态资源（如 Logo、轮播图等）。

这样做的最大好处是：AI 可以直接读取这个文件，知道当前项目有哪些功能，每个功能对应哪些代码文件。

具体例子包括：

- 在 REQ-1（用户认证模块）下，REQ-1-1（用户注册）和 REQ-1-2（用户登录）不仅描述了“注册成功要写入 users 表”“登录成功要建立会话”，还通过 `ui_spec` 详细规定了注册/登录页面的布局、输入校验规则和交互细节，并引用了对应的参考截图。
- 在 REQ-2（车票查询模块）下，REQ-2:SCE-0 详细说明了需要初始化的车站表、车次表及经停信息表，并约定北京、上海、南京三地之间至少 42 个车次的数据规模；REQ-2-2（车次列表页）及其子节点则拆分了 URL 参数解析、列表展示、预订按钮行为等多个场景。
- 在首页相关节点（例如 ROOT 的 `ui_spec`）中，写清楚了首页品牌区、导航栏、轮播图和快捷查询面板的结构和样式，并通过 `assets` 字段关联到实际使用的图片资源。

2.3 进度追踪

为了搞清楚项目做到了哪一步，我们在 `artifacts` 目录下维护了两个简单的辅助文件：

- `architecture_graph.yaml`：描述需求之间的父子关系，可以生成树状图。
- `phase_one_progress.yaml`：记录每个需求是“已处理”还是“未处理”，相当于一个任务进度表。

整体来看，我们的需求管理过程可以概括为四步：

1. 从已有代码和真实 12306 页面反推主流程；
2. 把这些流程整理成一棵带 ID 的需求树，并用 YAML 固化下来；
3. 在需求里嵌入具体的 UI 文件和接口路径，让需求、设计和实现对应得上；
4. 用 `artifacts` 目录下的图结构和进度文件，记录每个需求“做到什么程度了”，方便后续用智能体继续接力。

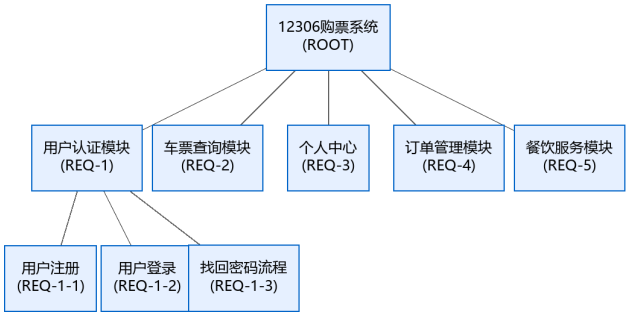


图 1: 需求结构图（示意）

3 需求规模

从需求树结构来看，我们在项目中覆盖了 12306 的核心业务流程，主要包括以下几层：

3.1 顶层模块数量

在新的 `requirements.yaml` 中，ROOT 下按功能拆出了多个大类模块，例如：

- **REQ-1 用户认证模块**：负责注册、登录、找回密码等功能；
- **REQ-2 车票查询模块**：负责车票数据初始化、首页快捷查询、车次列表页等；
- 其他模块：订单流程、个人中心、站车服务/餐饮等也分别作为独立的 REQ-x 节点存在。

3.2 子需求和场景数量

在顶层之下，我们又拆出若干子需求和具体场景，例如：

- 在 REQ-1（用户认证模块）下，拆出了 REQ-1-1（用户注册）、REQ-1-2（用户登录）、REQ-1-3-x（找回密码四个页面）等子节点，对应不同的页面和接口。
- 找回密码流程通过 REQ-1-3-1~4 四个节点覆盖了“填写账号信息—短信验证—设置新密码—完成提示”四步，每一步都有自己的 UI 规格和场景描述。
- 在 REQ-2（车票查询模块）下，REQ-2-1 系列节点拆分了首页快捷查询面板的各个输入（出发地、目的地、日期），REQ-2-2 系列节点覆盖了车次列表页初始化、从导航进入列表、点击“预订”等场景。

整个 YAML 文件里，每个需求节点下至少有 1 个场景，完整覆盖了用户从打开首页到完成购票、管理乘车人、下单餐饮的主要路径。相比只写“功能列表”，这样的拆分更贴近真实用户操作，也更方便后续写测试。

总体来看，我们没有刻意堆砌“需求点数量”，而是围绕一条完整业务链路，把关键节点拆细，并保证每个节点都有对应的 UI 页面和接口实现。

如果从“规模”的角度简单总结，可以认为：

- 顶层功能模块：1 个 ROOT + 若干个核心业务模块（认证、车票查询、订单、个人中心、站车服务/餐饮等）；
- 子需求：在认证、查询等模块下拆出了多级子节点（如注册、登录、快捷查询、车次列表等），方便精细化地描述每一块 UI 和交互；
- 场景数量：每个节点至少 1 个场景，从首页到购票、管理乘车人、以及扩展服务的完整路径都被覆盖到了。

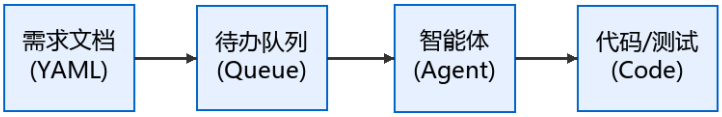


图 2: 智能体辅助开发流程

4 智能体辅助设计与测试

4.1 MCP 工具链的使用

为了让 AI 更好地帮助我们，我们引入了 MCP（Model Context Protocol）工具链。简单来说，就是通过几个脚本让 AI “听懂”我们的指令，以需求为入口去看代码：

- `init_top_down_queue`：读取需求 YAML，生成一个待办任务队列。
- `pop_next_requirement`：告诉 AI “下一个要做什么需求”。
- `register_ui_component`：告诉 AI “这个页面的 UI 组件在哪里，参数是什么”。

基于这套工具链，我们可以让 AI 按顺序沿着需求树“从 ROOT 走到各个 REQ 节点”（例如 REQ-1、REQ-1-1、REQ-2-2-1 等），每次只关注一个需求节点，减少遗漏。

4.2 AI 怎么帮我们干活

在实际开发中，AI 主要帮我们做了两件事：

1. **模块设计**：比如在做餐饮模块时，我们把需求告诉 AI，它会根据现有的代码风格，帮我们规划出需要新建哪些 Page 和 Component，甚至帮我们写出路由配置。
2. **测试辅助**：AI 知道每个需求的“期望结果”是什么，所以它可以帮我们生成测试思路。比如“在这个场景下，应该检查哪个按钮是否存在，或者哪个接口是否返回 200”。

4.3 如何保证可复现

为了让老师和助教能稳定复现我们的运行结果，我们把“数据初始化规则”也写进了需求里，让数据库可以**根据 YAML 自动生成**，而不是依赖某个固定快照。

- 在需求文档中，我们通过与数据初始化相关的场景（例如 REQ-2:SCE-0）约定了表结构、城市集合以及车次数量等约束；
- 实际运行时，由脚本读取这些约束，**随机生成**符合要求的车站和车次数据，不再提交单独的数据库文件。

这样，无论谁来运行我们的代码，只要按照相同的 YAML 需求执行初始化脚本，就能得到一份“结构和规模一致、满足约束”的数据库，用来完成查票和购票流程。对 AI 而言，也只需要先按需求文档中的初始化约定生成数据，再依次跑各个 UI 场景，就能复现实验结果。

5 测试用例与通过率

5.1 测试情况

我们主要进行了三类测试：

- **后端接口测试**：针对 Login、Query、Order 等核心 API，验证数据返回是否正确。
- **前端自动化测试**：使用前端测试框架跑单元/组件测试（最近一次运行结果记录在 `frontend/test-results/.last-run.json` 中，状态为 `passed`）。
- **界面功能测试**：手动验证了从登录到下单的完整流程，确保页面跳转逻辑通顺。

特别是在餐饮模块，我们重点测试了：

- 能否正确根据车次筛选商家；
- 商家列表是否展示正确；
- 点击商家能否进入详情页。

5.2 测试结果

从已经编写的自动化测试来看：

- **后端覆盖率**：整体语句覆盖率约为 56.75%，其中 `src/routes` 模块的语句覆盖率接近 98%，`src/models` 模块语句覆盖率超过 90%。核心路由和模型的基本分支都被覆盖到了。
- **前端测试结果**：最近一次前端测试的整体状态为 `passed`，当前已有的测试用例全部通过。

结合自动化与手工测试，可以认为在“正常使用路径”上的测试通过率较高，核心购票流程能够稳定跑通。

- **通过**：正常的用户购票流程、乘车人添加、餐饮浏览等主路径在当前测试集下均为通过状态。
- **待改进**：部分异常输入（如非法身份证号、极端日期）的处理还不够完善；部分脚本和工具类代码的覆盖率有提升空间；GUI 自动化测试覆盖的异常场景仍然偏少。

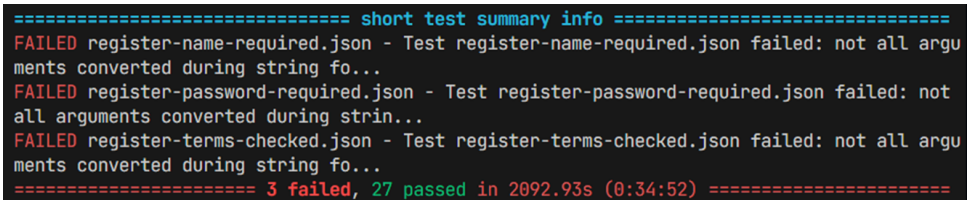


图 3: 测试执行结果截图

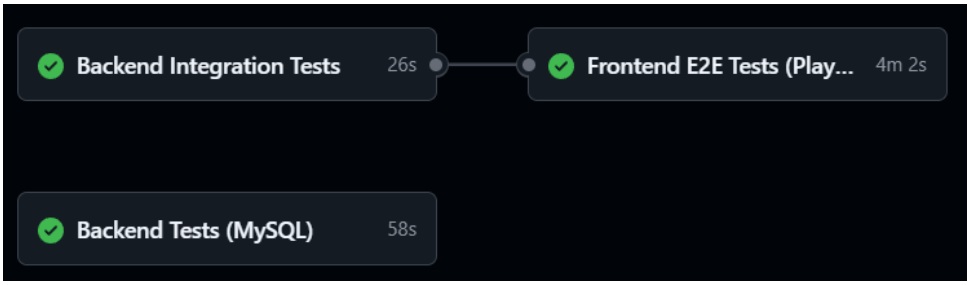


图 4: GitHub Actions CI 全部通过截图

6 总结

通过这个项目，我们不仅复刻了 12306 的核心功能，更重要的是实践了一种“文档驱动开发”的新模式。

- 需求不再是死文档，而是 AI 可读的 YAML；
- 数据库不再以快照形式提交，而是由脚本根据 YAML 中的约束自动生成，保证了环境的一致性；
- 智能体工具链让开发和测试变得更有条理。

这种方法让我们的项目不仅“能跑”，而且“好懂”、“好复现”。在此基础上，我们还特别强调了两点后续注意事项：

- 在生成或修改代码之后，要**补齐并运行端到端（E2E）测试**，不能只依赖 AI 的自然语言说明“已经通过”，而是要以真实的测试结果为准，确保所有代码版本都能稳定通过 E2E 测试；
- 在最终提交时，我们会**连同 YAML 需求文档和对应的 UI 截图一起交付**，其中 YAML 中会给出更详细的 UI 描述、数据初始化约束和截图引用，方便老师或智能体在没有源码上下文的情况下，仍然可以对照界面和数据规则来检查每个需求是否被满足，而数据库则由评测环境按同一套约束自动生成。